

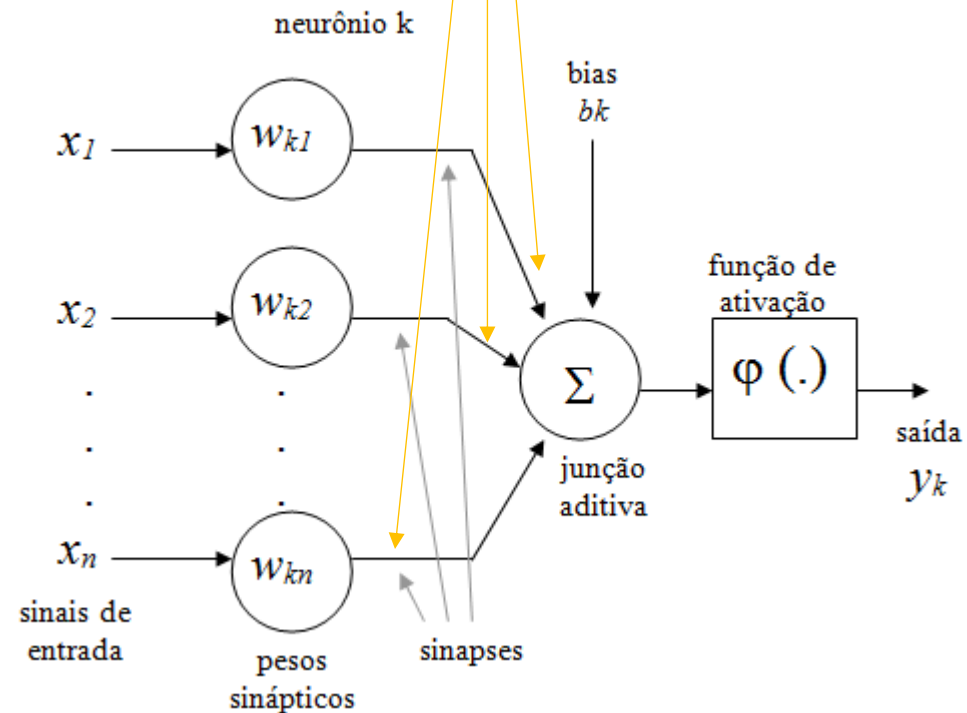


Redes Neurais Artificiais Aprendizado e algoritmo *Perceptron*

profº Mauricio Conceição Mario

Redes Neurais Artificiais: *Aprendizado*

- A etapa de aprendizado de uma RNA consiste de um processo iterativo de ajuste de parâmetros da rede, os **pesos das conexões**, que guardam, ao final do processo, o conhecimento que a rede adquiriu do ambiente externo.



Redes Neurais Artificiais: *Aprendizado*

- **pesos das sinapses w**
(pesos sinápticos)

vetor de pesos $\mathbf{w}(t + 1)$ no instante $t + 1$;
 $\mathbf{w}(t)$ = vetor de pesos no instante t ;
 $\Delta\mathbf{w}(t)$ = ajuste aplicado aos pesos.

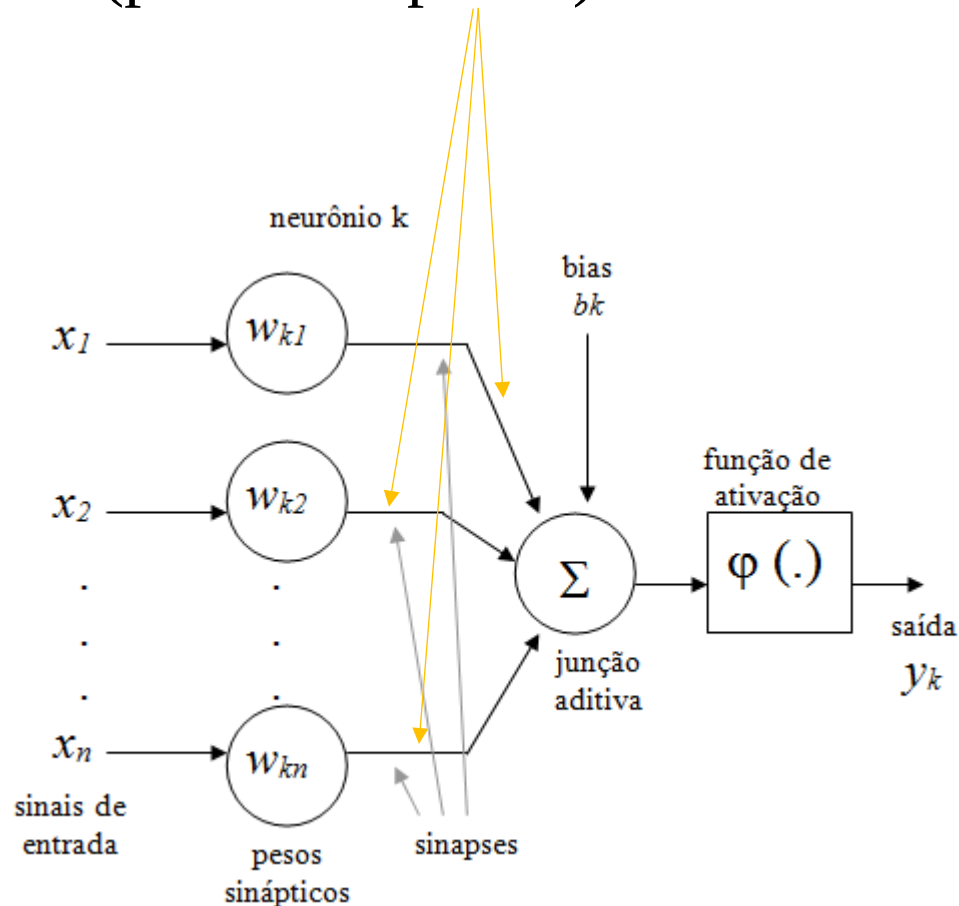
$$\mathbf{w}(t + 1) = \mathbf{w}(t) + \Delta\mathbf{w}(t)$$

Os algoritmos de aprendizado diferem na forma como $\Delta\mathbf{w}(t)$ é calculado.

Há algoritmos diferentes para treinamentos de Redes Neurais, podendo os mesmos serem agrupados em dois paradigmas principais:

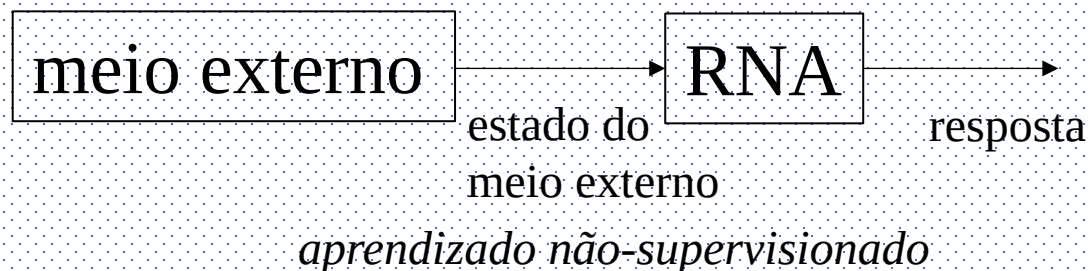
- *aprendizado supervisionado* e
- *aprendizado não-supervisionado*

(Braga, 2007)



Aprendizado não-supervisionado

- No aprendizado não-supervisionado não há a presença de um componente supervisor. Somente os padrões de entrada estão disponíveis para a rede. Durante o processo de aprendizado os padrões de entrada são apresentados continuamente à rede, e a existência de regularidades nesses dados faz com que o aprendizado seja possível. Se aplica a casos onde há regularidade e redundância nas entradas.



O aprendizado não-supervisionado se aplica a problemas que visam à descoberta de características estatisticamente relevantes nos dados de entrada, como, por exemplo, a descoberta de agrupamentos, ou de classes.

Aprendizado não-supervisionado → aprendizado *Hebbiano*

- A regra de aprendizado de **Hebb** (Hebb, 1949) propõe que o peso de uma conexão sináptica deve ser ajustado se houver sincronismo entre os “níveis de atividade” de entrada e saída. Se dois neurônios, em lados distintos de uma sinapse, são ativados sincronamente, tem-se um fortalecimento desta sinapse. Se estes tipos de neurônios citados são ativados assincronamente, a sinapse será enfraquecida ou mesmo eliminada.

$$\Delta w_{ij}(t) = \eta Y_i(t) x_j(t)$$

A interpretação do postulado de *Hebb* dentro do contexto das RNAs é que o ajuste dos pesos no tempo t deve ser proporcional ao produto dos valores de entrada e saída da rede (neurônio).

$w_{ij}(t)$ = valor do peso j do neurônio i ;

$x_j(t)$ = valor da entrada j ;

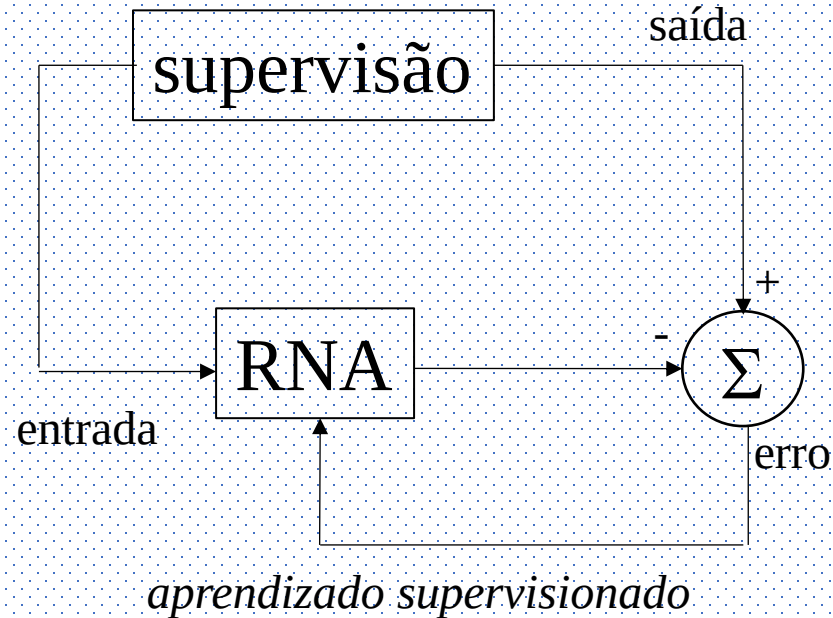
η = constante positiva que determina a taxa de aprendizado;

$Y_i(t)$ = saída do neurônio i ;



Redes Neurais Artificiais: Aprendizado Supervisionado

- Um supervisor é responsável por estimular as entradas da rede por meio de padrões de entrada e observar a saída calculada pela mesma, comparando com a saída desejada. Como a resposta da rede é função dos valores atuais do seu conjunto de pesos, estes são ajustados de forma a aproximar a saída da rede da saída desejada.



aprendizado supervisionado

(Braga, 2007)

O aprendizado supervisionado se aplica a problemas em que se deseja obter um mapeamento entre padrões de entrada e saída.

Os exemplos mais conhecidos de algoritmos para aprendizado são a *regra delta* e a sua generalização para redes de múltiplas camadas, o algoritmo *Backpropagation*.

Implementação ***off-line*** do aprendizado supervisionado: os dados do conjunto de treinamento não mudam, e, uma vez obtida uma solução para a rede, esta deve permanecer fixa.

Implementação ***on-line***: o conjunto de dados muda continuamente, e a rede deve estar em processo contínuo de adaptação.

Aprendizado Supervisionado: correção de erros

- No aprendizado supervisionado por correção de erros, procura-se minimizar o erro da resposta atual da rede em relação à saída desejada. A expressão genérica para o erro $e(t)$ no instante de tempo t pode ser escrita como:

$$e(t) = Y_d(t) - Y(t)$$

$w_i(t)$ = peso da entrada i ;

$x_i(t)$ = entrada i do neurônio;

η = taxa de aprendizado;

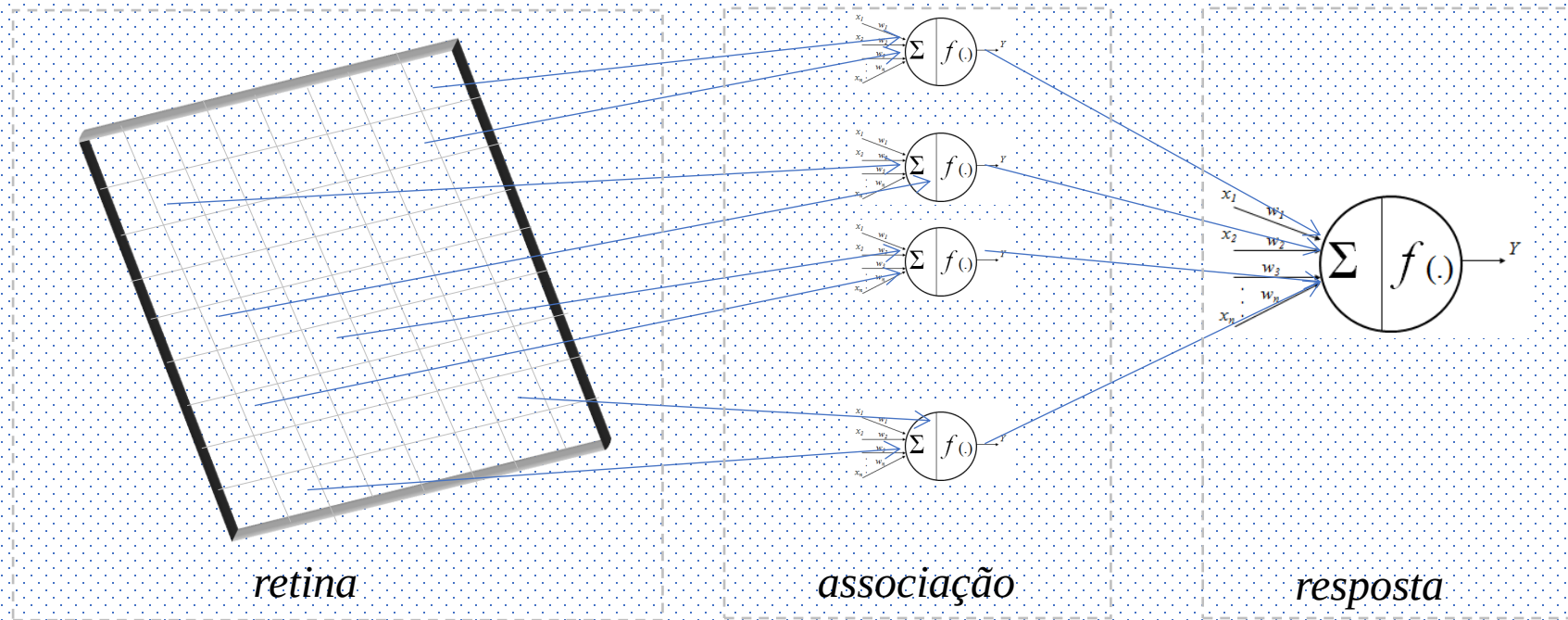
$e(t)$ = medida do erro;

$$w_i(t + 1) = w_i(t) + \overbrace{\eta e(t) x_i(t)}^{\Delta w(t)}$$

De acordo com a equação o ajuste de pesos deve ser proporcional ao produto do erro pelo valor da entrada da sinapse naquele instante de tempo.

O Perceptron

- Modelo proposto por Frank Rosenblatt composto por neurônios MCP e regra de aprendizado (Rosenblatt, 1962). A topologia original proposta tem unidades de entrada que formam a *retina*, um nível intermediário formado pelas *unidades de associação* e por um nível de saída formado pelas *unidades de resposta*.



(Braga, 2007)

Perceptron: algoritmo de aprendizado

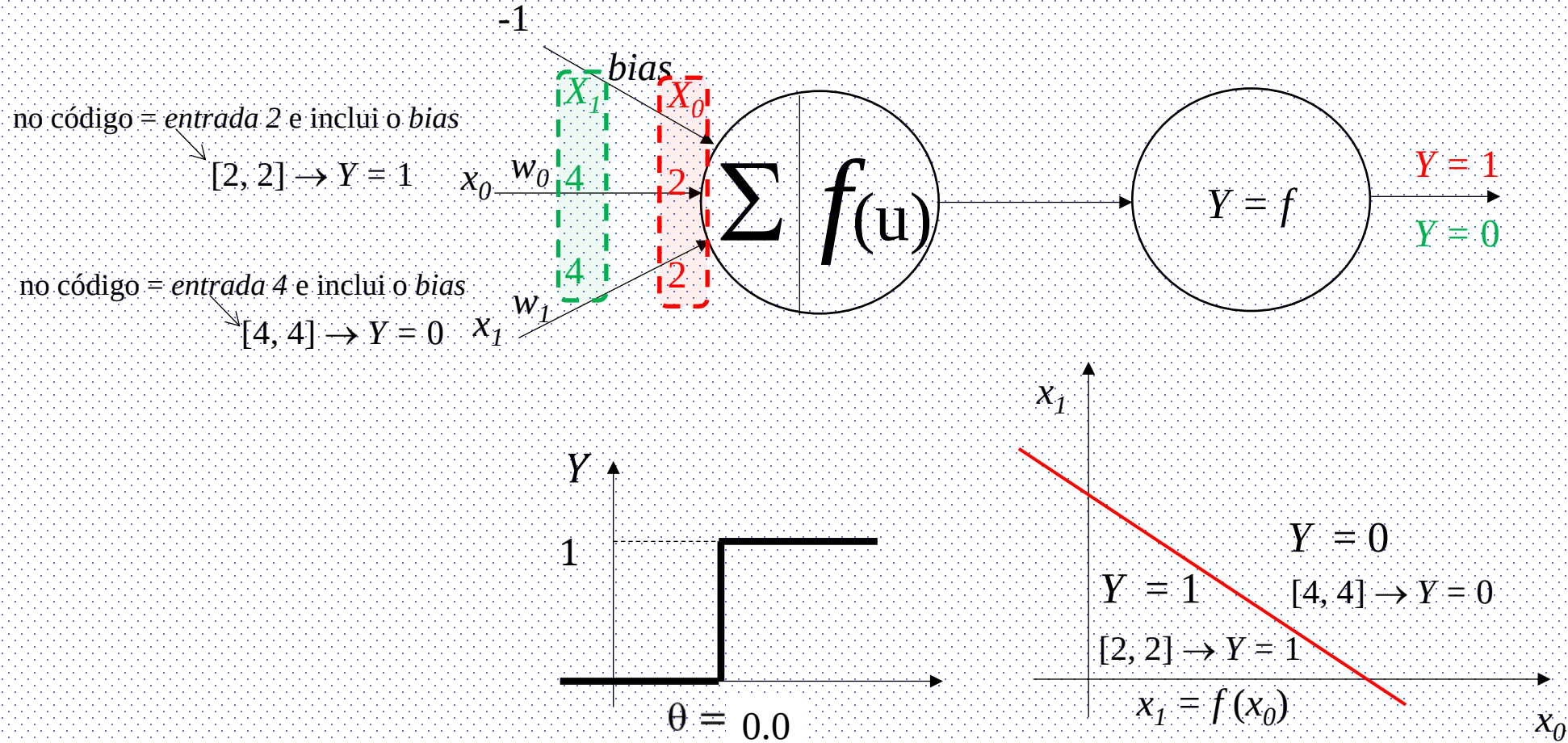
- $w(n)$ = vetor de pesos no instante n ;
- $\Delta w(n)$ = valor do incremento a ser aplicado ao vetor de pesos no instante n ;
- η = constante que é a medida da rapidez com que o vetor de pesos será atualizado;
- x = vetor de entrada;
- Y = saída atual;
- Yd = saída desejada;
- $e = Yd - Y \rightarrow$ erro.

“O algoritmo de treinamento de um *Perceptron* sempre chega, em um tempo finito, a uma solução para problemas de separação de duas classes linearmente separáveis.”

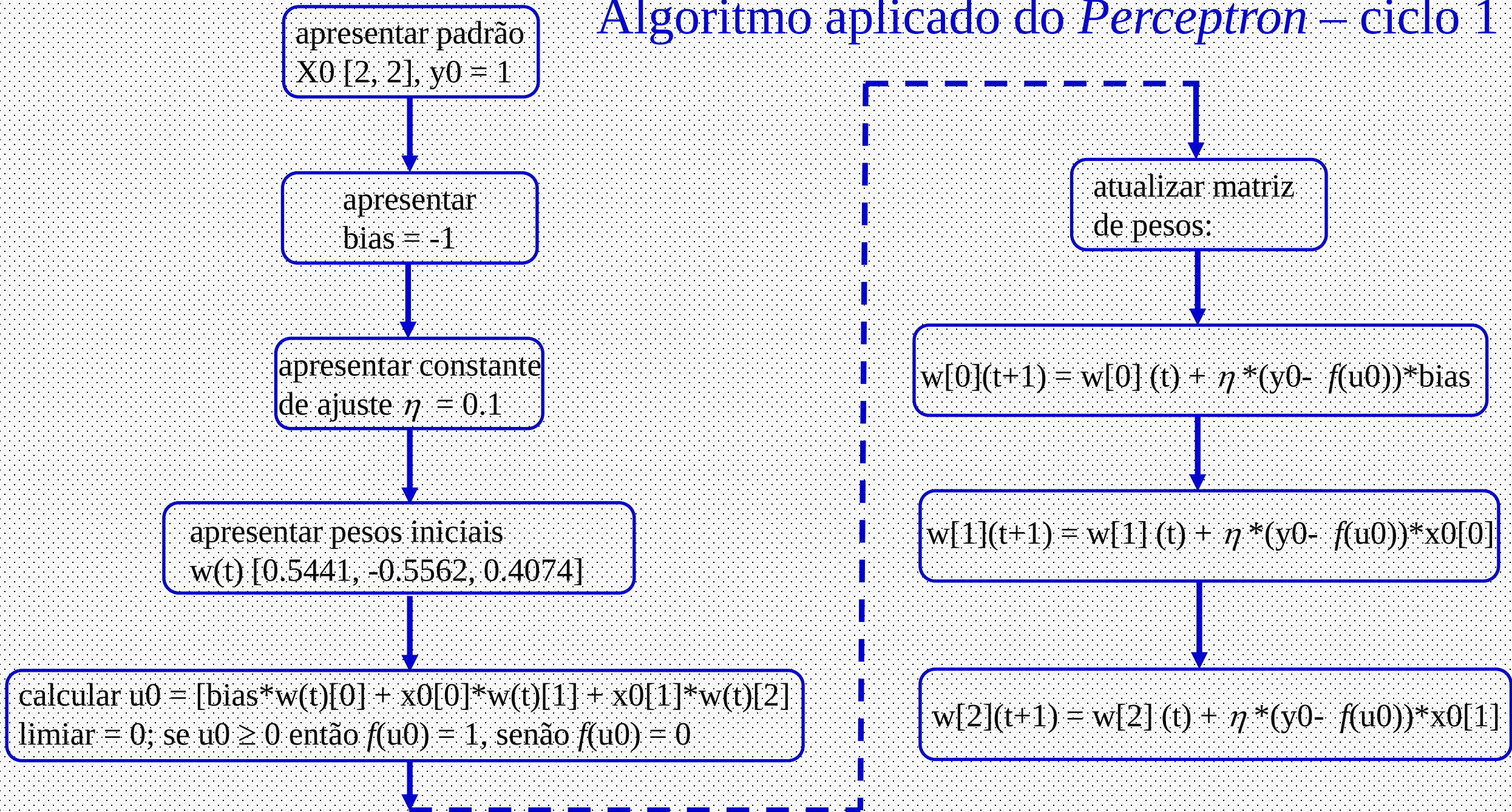
1. Inicializar o valor de η ;
2. Inicializar o vetor de pesos w com valores aleatórios;
3. Aplicar a regra de atualização dos pesos $w(n + 1) = w(n) + \eta e x(n)$ para todos os p pares (x_i, Yd_i) do conjunto de treinamento;
4. Repetir o passo 3 até que $e = 0$ para todos os p elementos do treinamento.

Uso do *Perceptron* para classificar padrões ou classes: exemplo

modelo da rede:



Algoritmo aplicado do *Perceptron* – ciclo 1



Algoritmo aplicado do *Perceptron* – ciclo 2

apresentar padrão
 $X1 [4, 4], y1 = 0$

apresentar
bias = -1

apresentar constante
de ajuste $\eta = 0.1$

apresentar pesos atualizados
 $w(t+1) [\dots, \dots, \dots]$

calcular $u0 = [\text{bias} * w(t+1)[0] + x1[0] * w(t+1)[1] + x1[1] * w(t+1)[2]$
limiar = 0; se $u0 \geq 0$ então $f(u0) = 1$, senão $f(u0) = 0$

atualizar matriz
de pesos:

$$w[0](t+2) = w[0](t+1) + \eta * (y1 - f(u0)) * \text{bias}$$

$$w[1](t+2) = w[1](t+1) + \eta * (y1 - f(u0)) * x1[0]$$

$$w[2](t+2) = w[2](t+1) + \eta * (y1 - f(u0)) * x1[1]$$

Algoritmo aplicado do *Perceptron* – ciclo 3

apresentar padrão
 $X0 [2, 2], y0 = 1$

apresentar
bias = -1

apresentar constante
de ajuste $\eta = 0.1$

apresentar pesos atualizados
 $w(t + 2) [\dots, \dots, \dots]$

calcular $u0 = [\text{bias} * w(t + 2)[0] + x0[0] * w(t + 2)[1] + x0[1] * w(t + 2)[2]$
limiar = 0; se $u0 \geq 0$ então $f(u0) = 1$, senão $f(u0) = 0$

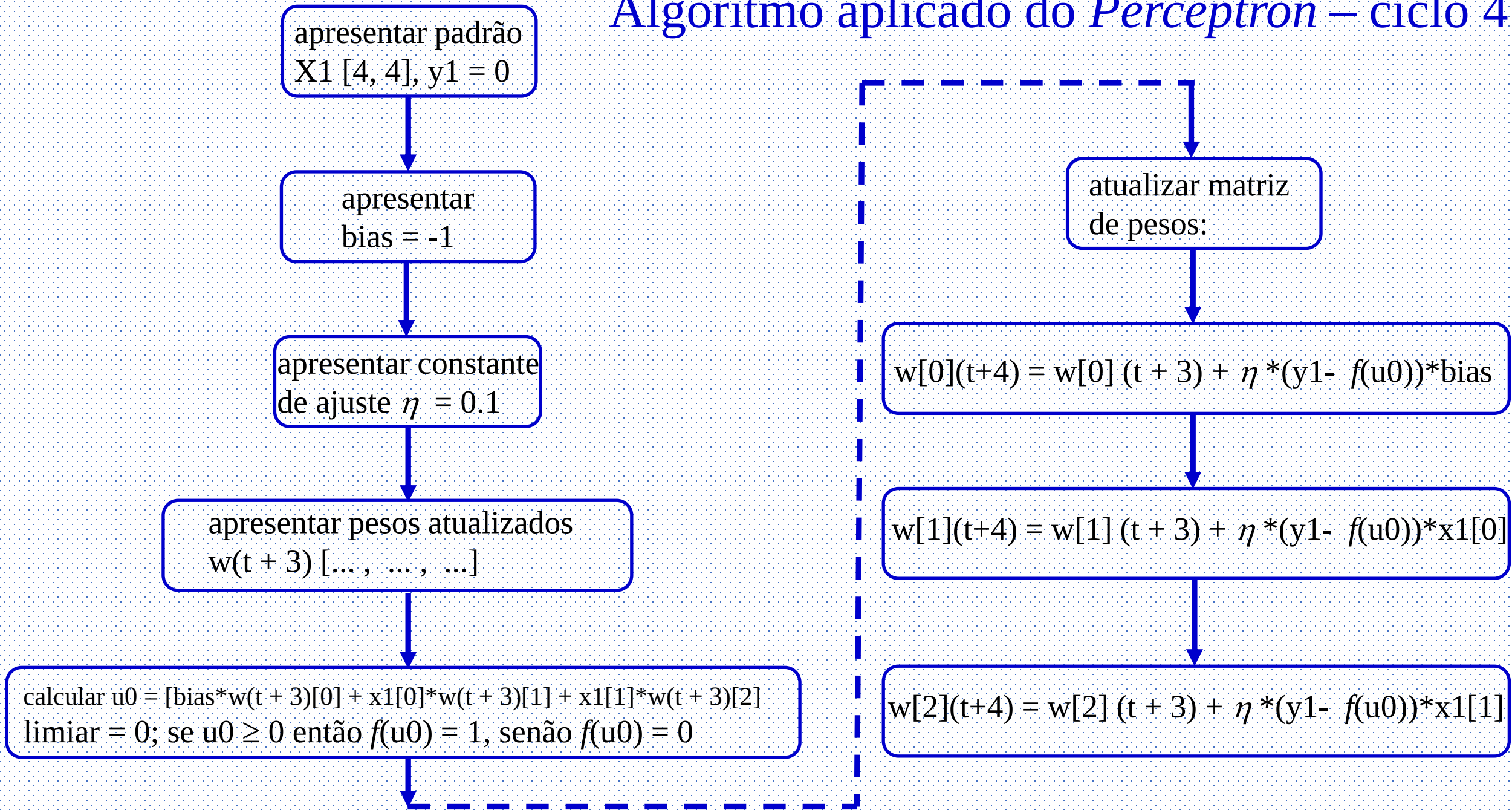
atualizar matriz
de pesos:

$$w[0](t+3) = w[0](t + 2) + \eta * (y0 - f(u0)) * \text{bias}$$

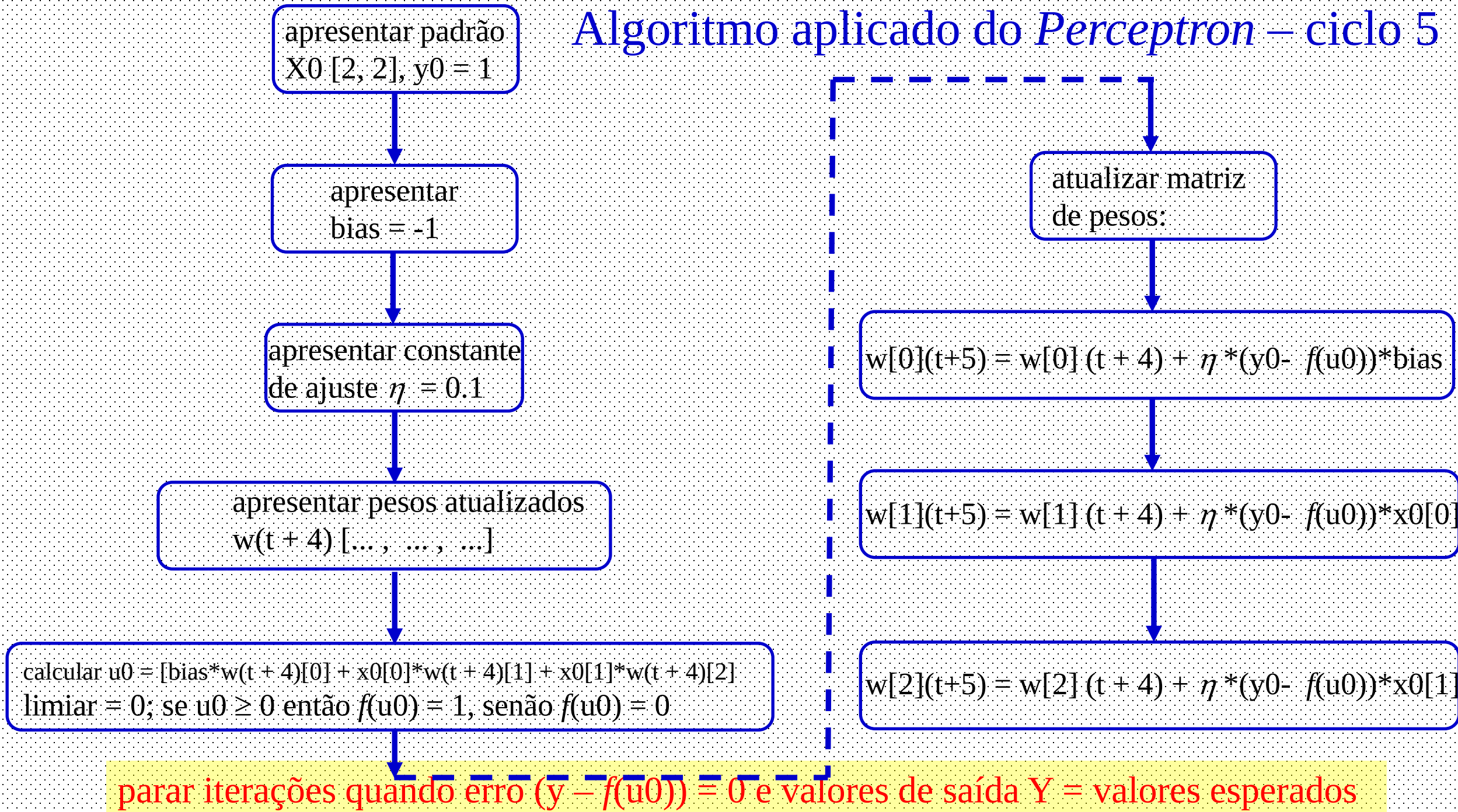
$$w[1](t+3) = w[1](t + 2) + \eta * (y0 - f(u0)) * x0[0]$$

$$w[2](t+3) = w[2](t + 2) + \eta * (y0 - f(u0)) * x0[1]$$

Algoritmo aplicado do *Perceptron* – ciclo 4



Algoritmo aplicado do *Perceptron* – ciclo 5



FileEditSearchSourceRunDebugConsolesProjectsToolsViewHelp

C:\Users\cmari\.spyder-py3

C:\Users\cmari\.spyder-py3\aprendizado.py

4.py x aplicação_perceptron.py x grafico_4.py x aproximação_funcional.py x senoide.py x adaline.py x perceptron_1.py x aprendizado.py x

```
8 from collections import Counter
9 from functools import partial
10 import matplotlib
11 import matplotlib.pyplot as plt"""
12 from linear_algebra import dot
13
14
15 def degrau(x):
16     return 1 if x >= 0 else 0
17
18 def saida_perceptron(pesos, entradas):
19     y = dot(pesos, entradas)
20     return degrau(y)
21
22 def ajustes(sinapses, entradas, saida):
23
24     taxa_aprendizagem = 0.1
25
26     saida_parcial = saida_perceptron(sinapses, entradas)
27
28     for j in range(3):
29         sinapses[j] = sinapses[j] + taxa_aprendizagem * (saida[0] - saida_parcial) * entradas[j]
30
31     saida = saida_parcial
32     return sinapses, saida
33
34
35 neuronio = [0.5441, -0.5562, 0.4074] # pesos das sinapses do neurônio
36 entrada2 = [-1, 2, 2]
37 entrada4 = [-1, 4, 4]
38 saida2 = [1]
39 saida4 = [0]
40 # ciclos de treinamento → 23
41 for _ in range(23):
42     neuronio, saida_2 = ajustes(neuronio, entrada2, saida2)
43     print (neuronio,"saida2 =", saida_2)
44     neuronio, saida_4 = ajustes(neuronio, entrada4, saida4)
45     print (neuronio,"saida4 =", saida_4)
46
47
48
```

SourceConsoleObject

Usage

Variable explorerHelpPlots

Console 1/A x

In [2]: runfile('C:/Users/cmari/.spyder-py3/aprendizado.py', wdir='C:/Users/cmari/.spyder-py3')
[0.44410000000000005, -0.3562, 0.6073999999999999] saida2 = 0
[0.5441, -0.7562, 0.20739999999999992] saida4 = 1
[0.44410000000000005, -0.5562, 0.40739999999999993] saida2 = 0
[0.44410000000000005, -0.5562, 0.40739999999999993] saida4 = 0
[0.34410000000000001, -0.3562, 0.6073999999999999] saida2 = 0
[0.44410000000000005, -0.7562, 0.20739999999999992] saida4 = 1
[0.34410000000000001, -0.5562, 0.40739999999999993] saida2 = 0
[0.34410000000000001, -0.5562, 0.40739999999999993] saida4 = 0
[0.24410000000000007, -0.3562, 0.6073999999999999] saida2 = 0
[0.24410000000000007, -0.5562, 0.40739999999999993] saida4 = 0
[0.14410000000000006, -0.3562, 0.6073999999999999] saida2 = 0
[0.24410000000000007, -0.7562, 0.20739999999999992] saida4 = 1
[0.14410000000000006, -0.5562, 0.40739999999999993] saida2 = 0
[0.14410000000000006, -0.5562, 0.40739999999999993] saida4 = 0
[0.044100000000000056, -0.3562, 0.6073999999999999] saida2 = 0
[0.14410000000000006, -0.7562, 0.20739999999999992] saida4 = 1
[0.044100000000000056, -0.5562, 0.40739999999999993] saida2 = 0
[0.044100000000000056, -0.5562, 0.40739999999999993] saida4 = 0
[-0.05589999999999995, -0.3562, 0.6073999999999999] saida2 = 0
[0.044100000000000056, -0.7562, 0.20739999999999992] saida4 = 1
[-0.05589999999999995, -0.5562, 0.40739999999999993] saida2 = 0
[-0.05589999999999995, -0.5562, 0.40739999999999993] saida4 = 0
[-0.15589999999999996, -0.3562, 0.6073999999999999] saida2 = 0
[-0.05589999999999995, -0.7562, 0.20739999999999992] saida4 = 1
[-0.15589999999999996, -0.5562, 0.40739999999999993] saida2 = 0
[-0.15589999999999996, -0.5562, 0.40739999999999993] saida4 = 0
[-0.25589999999999996, -0.3562, 0.6073999999999999] saida2 = 0
[-0.15589999999999996, -0.7562, 0.20739999999999992] saida4 = 1
[-0.25589999999999996, -0.5562, 0.40739999999999993] saida2 = 0
[-0.25589999999999996, -0.5562, 0.40739999999999993] saida4 = 0
[-0.3559, -0.3562, 0.6073999999999999] saida2 = 0
[-0.3559, -0.7562, 0.20739999999999992] saida4 = 1

IPython consoleHistory

LSP Python: readyKite: readyconda: base (Python 3.8.3)Line 41, Col 18UTF-8CRLFRWMem 65%

Digite aqui para pesquisar

09:5203/03/2021



C:\Users\cmari\spyder-py3\aprendizado.py

4.py x aplicação_perceptron.py x grafico_4.py x aproximação_funcional.py x senoide.py x adaline.py x perceptron_1.py x **aprendizado.py** x

```
8 from collections import Counter
9 from functools import partial
10 import matplotlib
11 import matplotlib.pyplot as plt"""
12 from linear_algebra import dot
13
14
15 def degrau(x):
16     return 1 if x >= 0 else 0
17
18 def saida_perceptron(pesos, entradas):
19     y = dot(pesos, entradas)
20     return degrau(y)
21
22 def ajustes(sinapses, entradas, saida):
23
24     taxa_aprendizagem = 0.1
25
26     saida_parcial = saida_perceptron(sinapses, entradas)
27
28     for j in range(3):
29         sinapses[j] = sinapses[j] + taxa_aprendizagem * (saida[0] - saida_parcial) * entradas[j]
30
31     saida = saida_parcial
32     return sinapses, saida
33
34
35 neuronio = [0.5441, -0.5562, 0.4074] # pesos das sinapses do neurônio
36 entrada2 = [-1, 2, 2]
37 entrada4 = [-1, 4, 4]
38 saida2 = [1]
39 saida4 = [0]
40 # ciclos de treinamento → 23
41 for _ in range(23):
42     neuronio, saida_2 = ajustes(neuronio, entrada2, saida2)
43     print (neuronio,"saida2 =", saida_2)
44     neuronio, saida_4 = ajustes(neuronio, entrada4, saida4)
45     print (neuronio,"saida4 =", saida_4)
46
47
48
```

Código do Aprendizado

Source Console Object

Usage

Variable explorer Help Plots

Console 1/A x

```
[-0.25589999999999996, -0.3562, 0.6073999999999999] saida2 = 0
[-0.15589999999999996, -0.7562, 0.20739999999999992] saida4 = 1
[-0.25589999999999996, -0.5562, 0.40739999999999993] saida2 = 0
[-0.25589999999999996, -0.5562, 0.40739999999999993] saida4 = 0
[-0.3559, -0.3562, 0.6073999999999999] saida2 = 0
[-0.2559, -0.7562, 0.20739999999999992] saida4 = 1
[-0.3559, -0.5562, 0.40739999999999993] saida2 = 0
[-0.3559, -0.5562, 0.40739999999999993] saida4 = 0
[-0.3559, -0.5562, 0.40739999999999993] saida2 = 1
[-0.3559, -0.5562, 0.40739999999999993] saida4 = 0
[-0.3559, -0.5562, 0.40739999999999993] saida2 = 1
[-0.3559, -0.5562, 0.40739999999999993] saida4 = 0
[-0.3559, -0.5562, 0.40739999999999993] saida2 = 1
[-0.3559, -0.5562, 0.40739999999999993] saida4 = 0
[-0.3559, -0.5562, 0.40739999999999993] saida2 = 1
[-0.3559, -0.5562, 0.40739999999999993] saida4 = 0
[-0.3559, -0.5562, 0.40739999999999993] saida2 = 1
[-0.3559, -0.5562, 0.40739999999999993] saida4 = 0
[-0.3559, -0.5562, 0.40739999999999993] saida2 = 1
[-0.3559, -0.5562, 0.40739999999999993] saida4 = 0
```

In [3]: |

IPython console History

LSP Python: ready

Kite: ready

conda: base (Python 3.8.3)

Line 41, Col 18

UTF-8

CRLF

RW

Mem 66%

Exercícios:

→ *editar e executar o código com o exemplo do algoritmo de aprendizado.*

→ *pesquisar um exemplo onde é utilizada uma Rede Neural Artificial com Aprendizado não-supervisionado.*

→ *pesquisar um exemplo onde é utilizada uma Rede Neural Artificial com Aprendizado supervisionado por reforço.*

Inserir as respectivas referências das pesquisas.

Referências Bibliográficas

- Braga AP, Carvalho APLF, Ludermir TB. *Redes Neurais Artificiais: teoria e aplicações*. Livros Técnicos e Científicos, Rio de Janeiro – RJ; 2007.
- Haykin S. *Neural Networks – A Comprehensive Foundation*. Prentice-Hall; 1994.
- Haykin S. *Redes Neurais – Princípios e prática*. 2a ed.. Porto Alegre: Bookman; 2001.
- Hebb DO. *The Organization of Behavior*. John Wiley & Sons; 1949.
- Heckerman D. *Probabilistic Similarity Networks*. MIT Press, Cambridge, Massachussets; 1991.
- Hopfield JJ. *Neurons with graded response have collective computational properties like those of two-state neurons*. Proceedings of the National Academy of Sciences of the United States of America, 79, 2554-2558; 1982.

Referências Bibliográficas

- Mario MC. *Proposta de Aplicação das Redes Neurais Artificiais Paraconsistentes como Classificador de Sinais Utilizando Aproximação Funcional*. Univ. Federal de Uberlândia, Dissertação de Mestrado, Uberlândia; 2003.
- McCarthy J. *Programs with common sense*. In Proceedings of the Symposium on Mechanisation of Thought Processes, Vol. 1, pp. 77-84, London. Her Majesty's Stationery Office; 1958.
- McCulloch W, Pitts W. *A logical calculus of the ideas immanent in nervous activity*. Bulletin of Mathematical Biophysics 5, 115-133; 1943.
- Rosenblatt F. *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*. Spartan, Chicago; 1962.
- Rumelhart DE, McClelland JL. *Parallel Distributed Processing*. MIT Press, Cambridge, Massachusetts; 1986.
- Turing A. *Computing machinery and intelligence*. Mind, 59, 433-460; 1950.